



Full-stack software development

Rendering

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Rendering

Rendering

- Terminology
 - “Rendering” is the process of turning your code into visible UI
 - “client” = the browser
 - “server” = the Next.js server, not our back-end api.
- How a page/component is rendered can be done in different ways:
 - Server-Side Rendering (SSR)
 - Static Site Generation (SSG)
 - Client-Side Rendering (CSR)

Server Rendering

- Remember: In Next.js, Layouts and Pages are React Server Components by default.
- On initial and subsequent navigations, the Server Component Payload is generated on the server before being sent to the client.

What is the React Server Component Payload (RSC)?

The RSC Payload is a compact binary representation of the rendered React Server Components tree.

Contains:

- The rendered result of Server Components
- Placeholders for Client Components references to their JavaScript files
- Any props passed from a Server Component to a Client Component

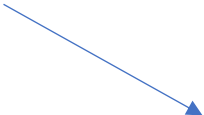


See slides about React Context:
how Next.js renders

Server Rendering

There are two types of server rendering, based on *when* it happens:

- **Static Rendering (or Prerendering)** happens at build time and the result is cached. (and when the cache needs updating)
 - Use this for pages that rarely change: an about page, the homepage in our demo app
- **Dynamic Rendering** happens at request time in response to a client request.



Terminology: don't mix these up

Server Components refer to *where* the code runs (on the server, not the browser), while Dynamic Rendering refers to *when* the code runs (at request time, not build time).

Server Rendering

- **Best Practice:** As we said before: keep components as Server Components whenever possible.
- Next.js will automatically choose Static Rendering (for speed) unless you use
 - dynamic features that require Dynamic Rendering, eg: cookies(), headers(), searchParams(),...
 - Explicit 'cache: no-store' in a fetch request: Next.js assumes the data changes constantly and will render the page dynamically

```
const response = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/lecturers`, {  
  method: 'GET',  
  headers: { ...  
},  
  cache: 'no-store',  
  credentials: 'include',  
});
```

Prefetching

This is an optimisation technique done by Next.js:

- Prefetching is the process of **loading a route in the background before the user navigates to it**. This makes navigation between routes in your application feel instant, because by the time a user clicks on a link, the data to render the next route is already available client side.

Prefetching

- Next.js automatically prefetches routes linked with the **<Link> component** but not the **<A> element**
- Next.js doesn't prefetch dynamic routes
(to prefetch dynamic routes, see info about streaming [here](#))
- You can opt out of streaming for a Link to save resources by setting prefetch to false

```
<Link prefetch={false} href="/blog">  
  Blog  
</Link>
```


Client-Side Rendering

- After the server has rendered the server components into a RSC payload & client components and that RSC payload are used to pre-render html
- On the first load on the client:
 - Prerendered HTML is shown (not interactive yet)
 - The RSC Payload is used to hydrate client components and make the application interactive

What is hydration

Hydration is React's process for attaching event handlers to the DOM, to make the static HTML interactive.

Client-Side Rendering

- On all subsequent navigations:
 - The RSC Payload is retrieved from cache for instant navigation
 - Client components are rendered entirely on the client with Javascript

Next.js Rendering Explained: From Server to Browser

